AI-Powered Ticket Classification System

Using Retrieval-Augmented Generation (RAG)

Graduation Project Documentation

Prepared in accordance with the Project Documentation Guidelines

Team Members:
Moustafa AbdElFattah Moustafa
Youssef Mohamed Kamel
Moustafa Ibrahim Moustafa
Seif Ahmed ElSayed
Aly Ehab Ahmed
Youssef Khaled Mohamed

Supervisor: Eng.Mohamed ElAbasairi

July 2026

1. Project Planning & Management

1.1 Project Proposal

Overview: Support teams triage large volumes of incoming customer tickets by hand, deciding a category, priority, and department before any work can begin. This project builds a web-based ticket management system that automates that first triage step using Retrieval-Augmented Generation (RAG): instead of training a dedicated classifier, the system embeds historical, already-labeled tickets, retrieves the most similar past tickets to a new one, and uses that retrieved evidence to decide the category, priority, department, and a suggested resolution.

Objectives:

- Automatically classify incoming support tickets into a fixed category set (Billing, Technical Support, Account Access / Management, Refund Request, Cancellation Request, Product Inquiry, General Inquiry).
- Ground every prediction in retrieved historical evidence so the decision is explainable, rather than a black-box label.
- Estimate ticket priority, urgency, and customer sentiment/emotion to help staff triage faster.
- Route each ticket to the correct department and an available employee, and notify both the customer and the employee.
- Let administrators correct wrong predictions, feeding that feedback back into future classifications (a lightweight form of continual learning).
- Provide role-based dashboards (customer, employee, administrator) with analytics on ticket volume, categories, priority, and sentiment.

Scope: The system covers ticket submission, AI-assisted classification and reply drafting, employee assignment, status tracking, an administrator control panel (user/employee management, feedback correction, analytics), and a conversational AI assistant. It does not include a public REST API, mobile clients, or a production-grade authentication system (see Section 6, Non-Functional Requirements, and the Risk Assessment below for the gaps this implies).

1.2 Project Plan

The project was delivered in the six phases required by the department's documentation guidelines. The table below is the working Gantt-style plan based on the graduate-track deadlines; update the dates/owners to match your actual submission track (student vs. graduate).

Phase	Key Deliverables	Milestone Date	Owner(s)
Project Planning & Management	Proposal, plan, roles, risk register, KPIs	1/27/2026	[Team]
Literature Review	RAG / helpdesk automation research, feedback	3/1/2026	[Team]
Requirements Gathering	Stakeholder analysis, use cases, FRs/NFRs	3/1/2026	[Team]
System Analysis & Design	Architecture, ERD, DFD, sequence/class diagrams, UI	4/3/2026	[Team]
Implementation	RAG pipeline, Streamlit app, database, AI assistant	5/17/2026	[Team]
Final Presentation & Testing	Test cases, user manual, tech docs, demo	5/24/2026	[Team]

Resource allocation: the working codebase separates cleanly along the pipeline shown in Section 4.3 (embedding, retrieval, classification, NLP enrichment, persistence, UI), which is a natural way to split work across a 3–5 person team — e.g. one person on the RAG/embedding core, one on the Streamlit UI and dashboards, one on the database/auth layer, and one on NLP add-ons (sentiment, emotion, explainability) and the AI assistant.

1.3 Task Assignment & Roles

Role	Responsibilities	Team Member
RAG / ML Engineer	Embedding backends, FAISS vector store, classification & voting logic (src/embedder.py, vector_store.py, rag_classifier.py)	Moustafa AbdElFattah
Backend / Database Engineer	SQLite schema, auth, ticket/feedback/notification persistence (src/database.py)	Moustafa Ibrahim
Frontend / UI Engineer	Streamlit app, role-based dashboards, analytics charts (app.py, style.css)	Seif Ahmed
NLP / AI Features Engineer	Sentiment, emotion, keyword-based explainability, Gemini assistant (src/sentiment.py, emotion.py, explainable_ai.py, gemini_ai.py)	Youssef Mohamed Aly Ehab
QA / Documentation Lead	Test cases, dataset preparation, documentation, presentation	Youssef Khaled

1.4 Risk Assessment & Mitigation Plan

Risk	Likelihood	Impact	Mitigation
Cold-start: too few labeled historical tickets for a category → poor retrieval matches	Medium	High	Grow data/historical_tickets_clean.csv (2,680 labeled rows already collected); require a minimum similarity threshold (0.60) before treating a match as confident, and fall back to similarity voting otherwise
Dependence on optional external APIs (OpenAI, Gemini) for LLM classification and the AI assistant	Medium	Medium	System auto-detects API keys and gracefully falls back to the fully offline similarity-voting classifier when no key is set
Passwords stored in plaintext in the users/employees tables	High (currently true)	High	Before any real deployment, hash credentials (bcrypt is already a project dependency via src/auth.py) and remove the hardcoded default admin/employee passwords in src/database.py
Heavy ML dependencies (sentence-transformers, transformers, keybert) increase install size and first-run latency (model downloads)	Medium	Medium	Document exact versions in requirements.txt; consider caching downloaded models in CI/deployment images

No automated test suite — test.py is a manual ad-hoc script, not a repeatable test	High (currently true)	Medium	Add pytest-based unit tests for the classifier, database layer, and edge cases (see Section 6)
Single local SQLite file (users.db) is not safe for concurrent multi-user production traffic	Low for a coursework demo, High for production	Medium	Acceptable for the graduation-project scope; document as a known limitation and a future-work item (see README 'Ideas for Extending This')

1.5 KPIs (Key Performance Indicators)

- Classification accuracy — agreement between predicted category and the true category on a held-out test split of data/historical_tickets_clean.csv (2,680 labeled tickets across Billing, Technical Support, Account Management, and General Inquiry).
- Retrieval confidence — average top-1 cosine similarity score returned by FAISS; the system already treats similarity ≥ 0.60 as a 'historical match' internally.
- Response/processing time — end-to-end time from ticket submission to displayed classification (embedding + FAISS search + optional LLM call).
- System uptime — availability of the Streamlit application during the demo/grading window.
- User adoption / feedback-correction rate — proportion of tickets an administrator has to manually re-categorize via the feedback tool (src/database.py: save_feedback); a decreasing rate over time indicates the feedback loop is improving predictions.
- Employee workload balance — distribution of assigned tickets across employees per department, visible on the Admin Dashboard.

2. Literature Review

2.1 Background & Related Approaches

Automated ticket triage is a well-established application of NLP in IT service management and customer support. Two broad approaches are commonly compared in the literature and in production systems:

- Supervised classifiers (e.g., TF-IDF/SVM, fine-tuned BERT) trained end-to-end on labeled tickets. These can reach high accuracy but need retraining whenever categories or label definitions change, and they give little insight into why a label was chosen.
- Retrieval-Augmented Generation (RAG), the approach used in this project: new text is matched against a vector index of already-labeled examples, and the label is derived from the retrieved neighbors (via similarity voting or an LLM reasoning over them). This trades some raw accuracy for adaptability (new labeled tickets can be added without retraining) and explainability (the exact matching historical tickets are shown as evidence).

This project's design mirrors how RAG is used in production customer-support automation, IT helpdesk triage, and smart email routing (as noted in the project README), while keeping a fully offline fallback path so the system does not have a hard dependency on a paid LLM API.

2.2 Feedback & Evaluation

[To be completed by the supervising lecturer after review.]

Criterion	Comments	Score
Technical soundness of the RAG approach		
Code quality and structure		
Documentation completeness		
Presentation & demo		

2.3 Suggested Improvements

Based on a review of the current implementation, in addition to the extensions already listed in the project README ('Ideas for Extending This'), the following improvements would strengthen the system:

- Replace plaintext password storage with bcrypt hashing (the dependency already exists in src/auth.py, but the active database module — src/database.py — does not use it).
- Reconcile the two parallel database modules (src/auth.py and src/database.py); only one (database.py) is actually wired into app.py, and the other appears to be an earlier iteration left in the repository.
- Add a proper automated test suite (unit tests for the classifier, retrieval, and database functions) — see Section 6.
- Update requirements.txt, which currently lists only the original TF-IDF/FAISS/OpenAI stack and is missing streamlit, transformers, sentence-transformers, keybert, bcrypt, google-genai, and matplotlib, all of which the running application depends on.
- Add an evaluation script that reports precision/recall/F1 per category on a held-out split, to turn the 'classification accuracy' KPI into a reproducible number.
- Consider sentence-transformers or OpenAI embeddings as the default backend for graded submissions (already implemented and used by main.py's demo command) instead of plain TF-IDF, since the semantic embedder captures paraphrased tickets that share no exact keywords.

2.4 Final Grading Criteria

Component	Weight (suggested)
Documentation (this document + README)	20%
Implementation (source code, functionality)	35%
Testing	15%
Presentation & Demo	20%
Innovation / extensions beyond baseline requirements	10%

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Interest / Need
Customer (end user submitting tickets)	Fast, accurate categorization; clear reply and expected resolution time; visibility into ticket status
Support Employee	A manageable, correctly-routed queue of tickets matching their department; ability to update status and close tickets
Administrator	Oversight of all tickets and users; ability to correct AI mistakes; analytics on volume, categories, and sentiment; user/employee management
Course Supervisor / Grader	Clear documentation, working demo, evidence of sound engineering practice

3.2 User Stories & Use Cases

- As a customer, I want to describe my problem in free text and get an immediate predicted category and suggested next steps, so I don't have to guess which department to contact.
- As a customer, I want to see similar past tickets and their outcomes, so I trust that the system's suggestion is grounded in real precedent, not a guess.
- As a customer, I want to ask follow-up questions to an AI assistant about my ticket, so I can get more context without waiting for a human.
- As an employee, I want to see only the tickets assigned to my department and me, sorted by priority, so I can work through my queue efficiently.
- As an employee, I want to mark a ticket as in-progress or finished, so the customer and the admin see up-to-date status.
- As an administrator, I want to correct a wrong category, so that similar future tickets benefit from that correction.
- As an administrator, I want dashboards of ticket volume by category, priority, and sentiment, so I can spot trends and staffing needs.
- As an administrator, I want to manage user and employee accounts (add, remove, change status), so I can control who has access to the system.

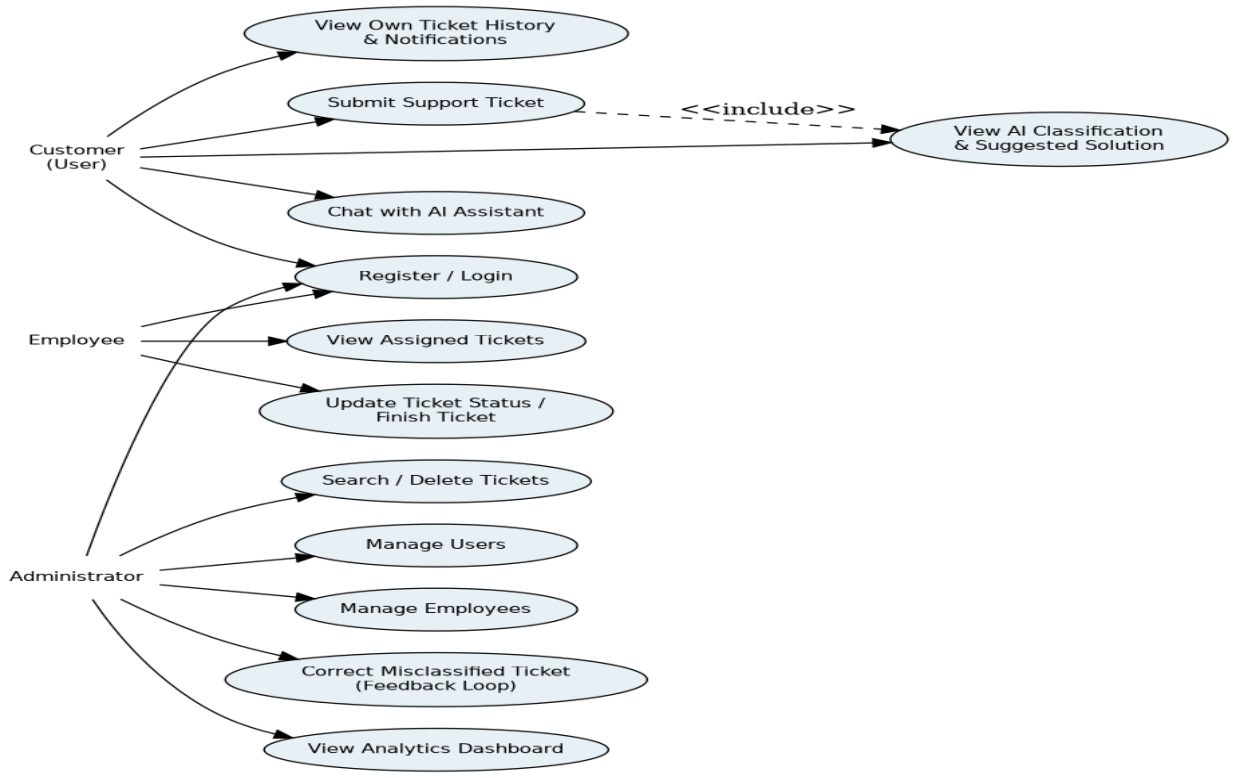

Figure 3.1 — Use Case Diagram: Customer, Employee, and Administrator actors and their interactions with the system.

3.3 Functional Requirements

- FR1 — The system shall allow users to register and log in as a customer, and shall separately authenticate employees and administrators (src/database.py: register_user, login_user, login_employee).
- FR2 — The system shall accept free-text ticket descriptions and return a predicted category, priority, department, estimated resolution time, and confidence score.
- FR3 — The system shall retrieve and display the top-k most similar historical tickets as supporting evidence for the prediction (RAGTicketClassifier.retrieve).
- FR4 — The system shall detect customer sentiment (Positive/Negative) and emotion using pretrained NLP models, and shall estimate urgency from keyword patterns (Critical/High/Medium/Low).
- FR5 — The system shall produce an explainability summary: extracted keywords, the reasons behind the decision, and agreement level among retrieved neighbors (src/explainable_ai.py).
- FR6 — The system shall automatically assign each ticket to an available employee in the correct department (src/assignment.py, database.assign_employee).
- FR7 — The system shall notify the assigned employee and the submitting customer of relevant status changes (src/database.py: add_notification, get_notifications).
- FR8 — The system shall let an administrator correct a misclassified ticket's category, storing the correction for future retrieval (save_feedback), and future classifications shall prefer a sufficiently similar past correction over a fresh prediction (RAGTicketClassifier.search_feedback, similarity ≥ 0.85).
- FR9 — The system shall provide an administrator dashboard to manage users and employees (create, delete, change status) and to search/delete/update the status of any ticket.
- FR10 — The system shall provide analytics visualizations: tickets by category, priority distribution, sentiment distribution, urgency distribution, and tickets per day.

- FR11 — The system shall provide a conversational AI assistant (Google Gemini) that can discuss a ticket's analysis and answer free-form questions about it.

3.4 Non-Functional Requirements

- Performance — a classification request should complete within a few seconds for interactive use; FAISS's IndexFlatIP gives exact nearest-neighbor search suitable for the current dataset scale (tens to a few thousand tickets).
- Usability — the interface must be usable without technical training; role-appropriate views are shown automatically after login (customer / employee / admin).
- Availability of core functionality without external dependencies — the classifier must work fully offline (TF-IDF or local sentence-transformers embeddings + similarity voting) when no OpenAI/Gemini API key is configured.
- Explainability — every prediction must be traceable to the retrieved evidence tickets that produced it, not just a bare label.
- Security (current gap — see Section 1.4) — credentials should be hashed, not stored in plaintext, before any deployment beyond local grading/demo use.
- Maintainability — the pipeline is modular (embedder / vector store / classifier / database / UI are separate modules), so a component (e.g., the embedding backend) can be swapped without touching the rest of the system.
- Portability — the system should run locally via `streamlit run app.py` with only a Python environment and the packages in requirements.txt (once corrected — see Section 2.3).

4. System Analysis & Design

4.1 Problem Statement & Objectives

Problem: manually triaging incoming support tickets (deciding category, urgency, and the right team) is slow, inconsistent between agents, and does not scale with ticket volume. Objective: build a system that classifies each ticket automatically, grounds that classification in real historical precedent (for trust and explainability), enriches it with sentiment/urgency signals, routes it to the right department and employee, and lets humans correct mistakes in a way the system learns from.

4.1.1 Use Case Diagram & Descriptions

See Figure 3.1 above. Brief descriptions:

- Submit Support Ticket (Customer) — includes the AI classification use case; every submission is automatically analyzed.
- View Assigned Tickets / Update Ticket Status (Employee) — an employee only sees tickets routed to their department and to them personally.
- Correct Misclassified Ticket / Manage Users / Manage Employees / View Analytics (Administrator) — administrative use cases layered on top of the same ticket data.

4.1.2 Functional & Non-Functional Requirements

Restated in full in Sections 3.3 and 3.4 above.

4.1.3 Software Architecture

The system follows a layered architecture: a Streamlit presentation layer, an application/intelligence layer that implements the RAG pipeline and auxiliary NLP services, and a data layer (SQLite for relational data, a FAISS index + pickled metadata for the vector store, and CSV files for the historical training data).

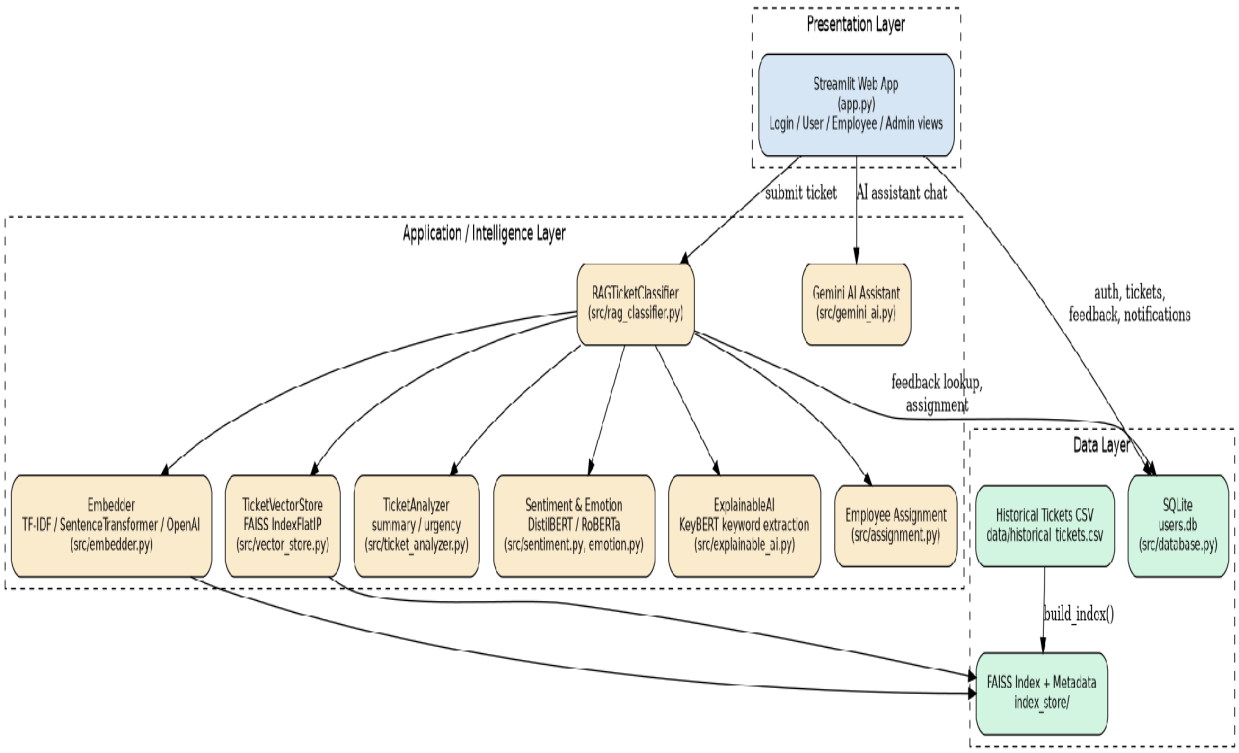

Figure 4.1 — High-level software architecture.

4.2 Database Design & Data Modeling

4.2.1 ER Diagram

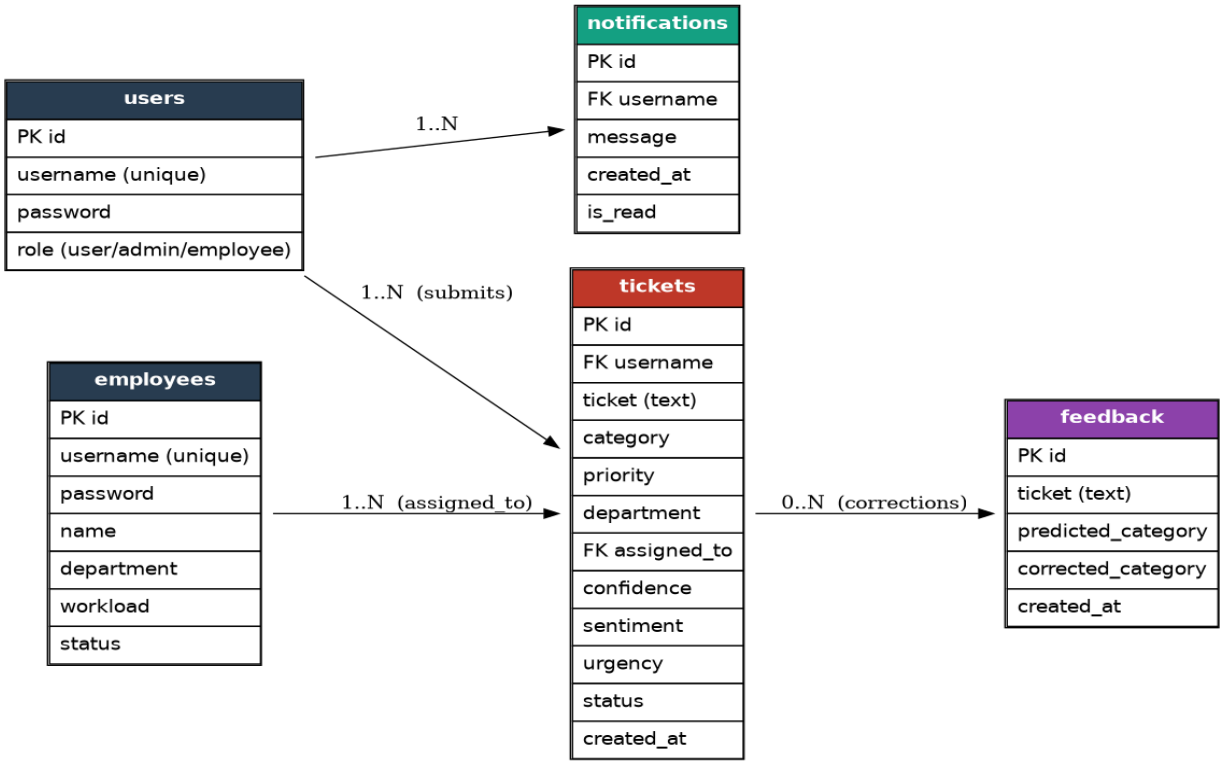

Figure 4.2 — Entity-Relationship Diagram (SQLite, users.db).

4.2.2 Logical & Physical Schema

users — application accounts used for login and role checks.

Column	Type	Notes
id	INTEGER PK	Autoincrement
username	TEXT	Unique
password	TEXT	Currently stored in plaintext — see Section 1.4 risk
role	TEXT	'user' (default), 'admin', or 'employee'

employees — profile data for staff, separate from their users login row.

Column	Type	Notes
id	INTEGER PK	Autoincrement
username	TEXT	Unique; matches a users row with role='employee'
password	TEXT	Plaintext (see risk note above)

name	TEXT	Display name
department	TEXT	e.g. Finance, Technical Support, Account
workload	INTEGER	Default 0
status	TEXT	Default 'Available'

tickets — every submitted ticket and its AI analysis.

Column	Type	Notes
id	INTEGER PK	Autoincrement
username	TEXT	FK → users.username (submitter)
ticket	TEXT	Raw customer text
category / priority / department	TEXT	AI-assigned classification output
assigned_to	TEXT	FK → employees.name
confidence	REAL	Similarity / model confidence, 0–1
sentiment / urgency	TEXT	NLP-derived signals
status	TEXT	e.g. Open, In Progress, Finished
created_at	TEXT	Timestamp

feedback — administrator corrections used to improve future predictions.

Column	Type	Notes
id	INTEGER PK	Autoincrement
ticket	TEXT	Original ticket text
predicted_category / corrected_category	TEXT	Before/after correction
created_at	TEXT	Timestamp

notifications — per-user alerts (e.g., ticket assigned, status changed).

Column	Type	Notes
id	INTEGER PK	Autoincrement
username	TEXT	FK → users.username (recipient)
message / created_at / is_read	TEXT / TEXT / INTEGER	Notification content and read state

4.3 Data Flow & System Behavior

4.3.1 DFD (Data Flow Diagram)

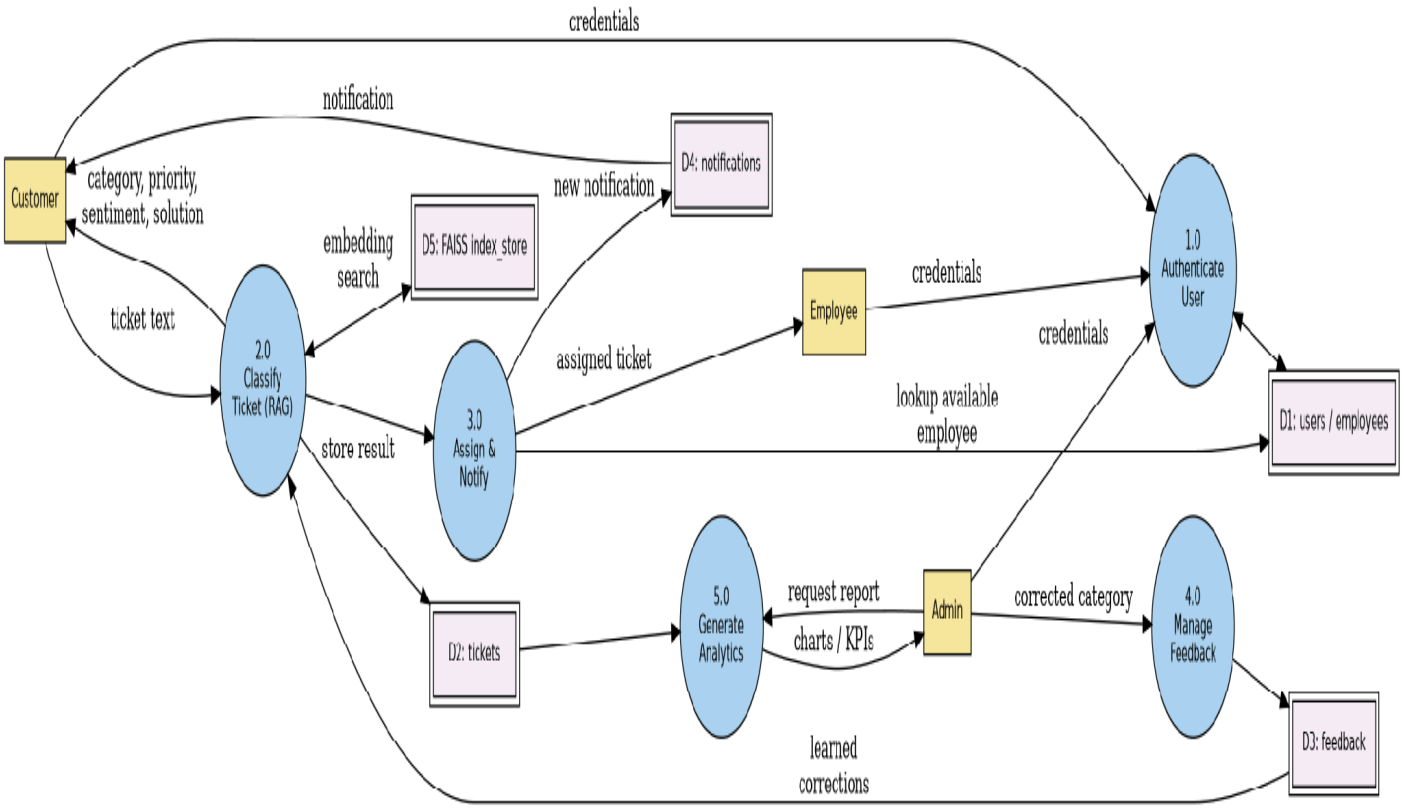

Figure 4.3 — Data Flow Diagram: authentication, classification, assignment/notification, feedback, and analytics processes.

4.3.2 Sequence Diagram — Ticket Classification

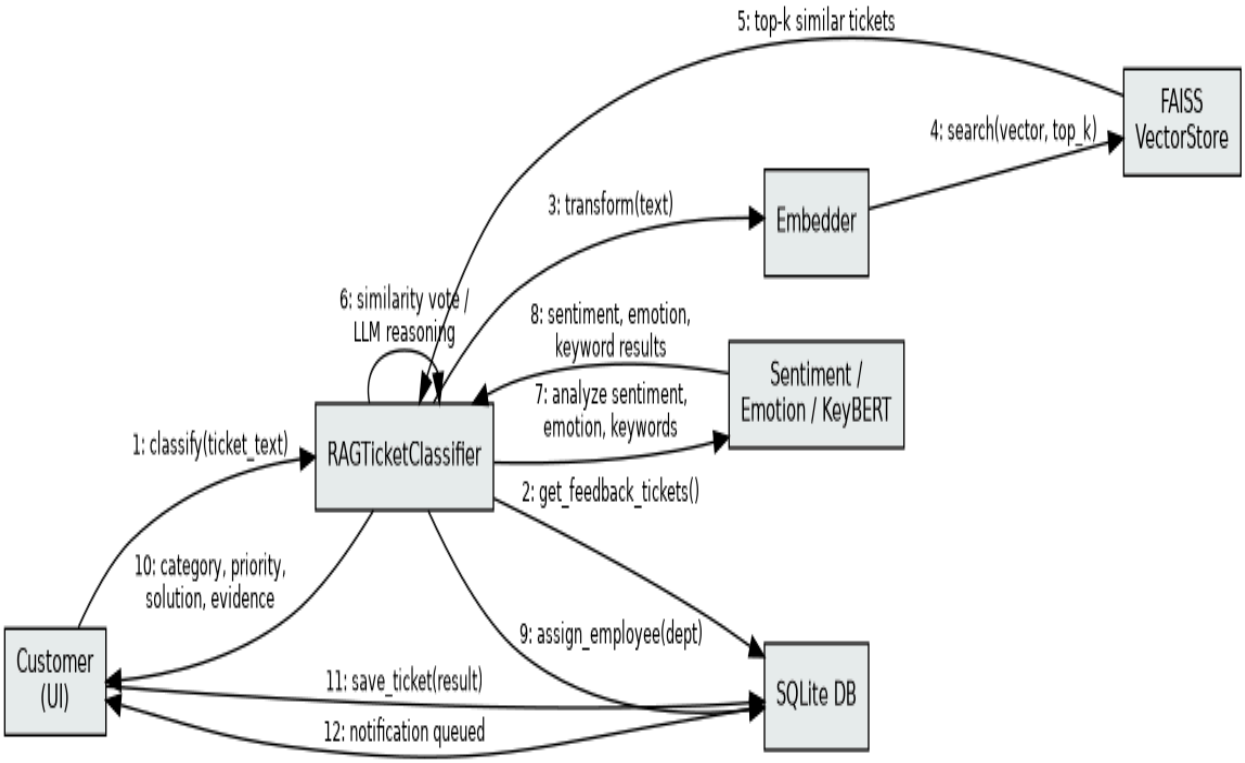

Figure 4.4 — Sequence of calls when a customer submits a new ticket (RAGTicketClassifier.classify).

4.3.3 Activity Diagram — Ticket Lifecycle

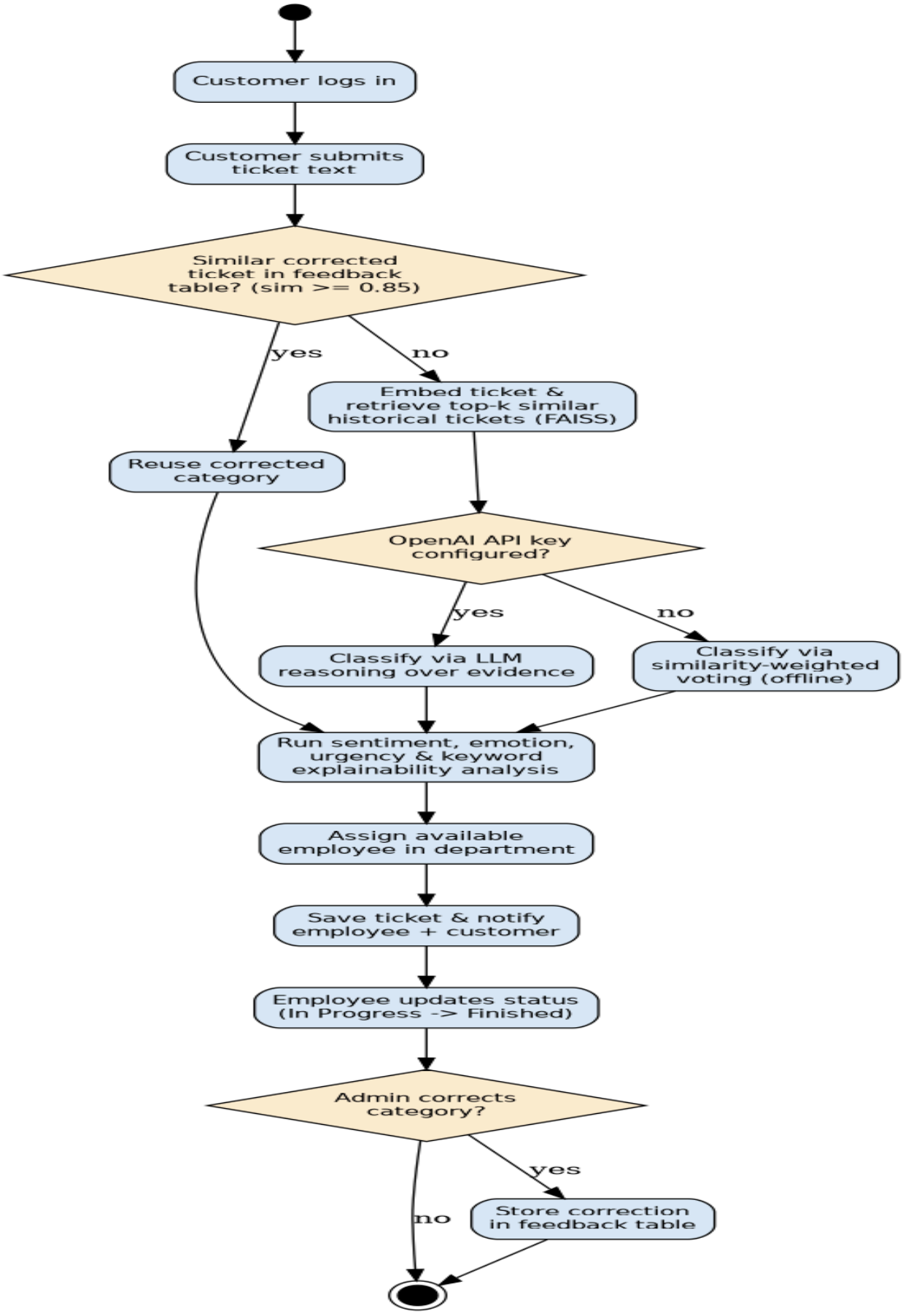
Customer logs in
Customer submits ticket text
Similar corrected ticket in feedback table? (sim >= 0.85)
yes
no
Reuse corrected category
Embed ticket & retrieve top-k similar historical tickets (FAISS)
OpenAI API key configured?
yes
no
Classify via LLM reasoning over evidence
Classify via similarity-weighted voting (offline)
Run sentiment, emotion, urgency & keyword explainability analysis
Assign available employee in department
Save ticket & notify employee + customer
Employee updates status (In Progress -> Finished)
Admin corrects category?
yes
no
Store correction in feedback table

Figure 4.5 — Activity Diagram: full decision flow from ticket submission to employee assignment and optional admin correction.

4.3.4 State Diagram — Ticket Status

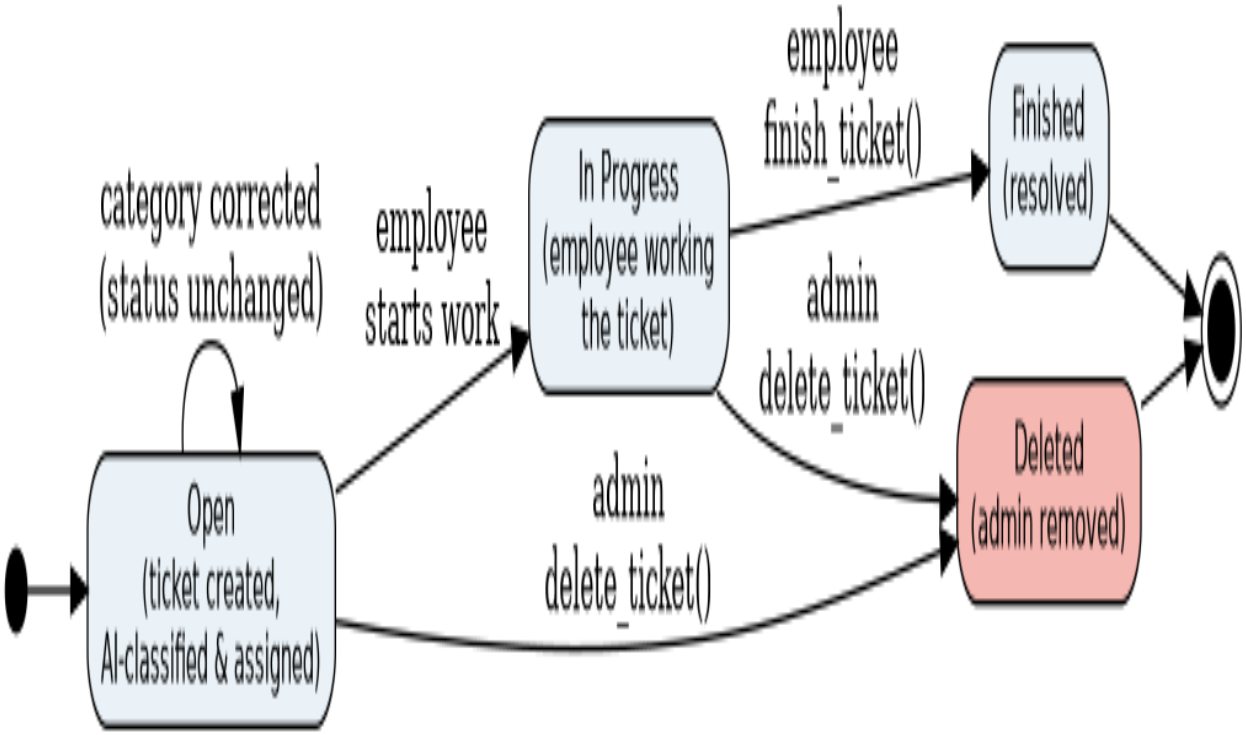

Figure 4.6 — State Diagram: ticket status transitions (Open → In Progress → Finished, with deletion possible at any state).

4.3.5 Class Diagram

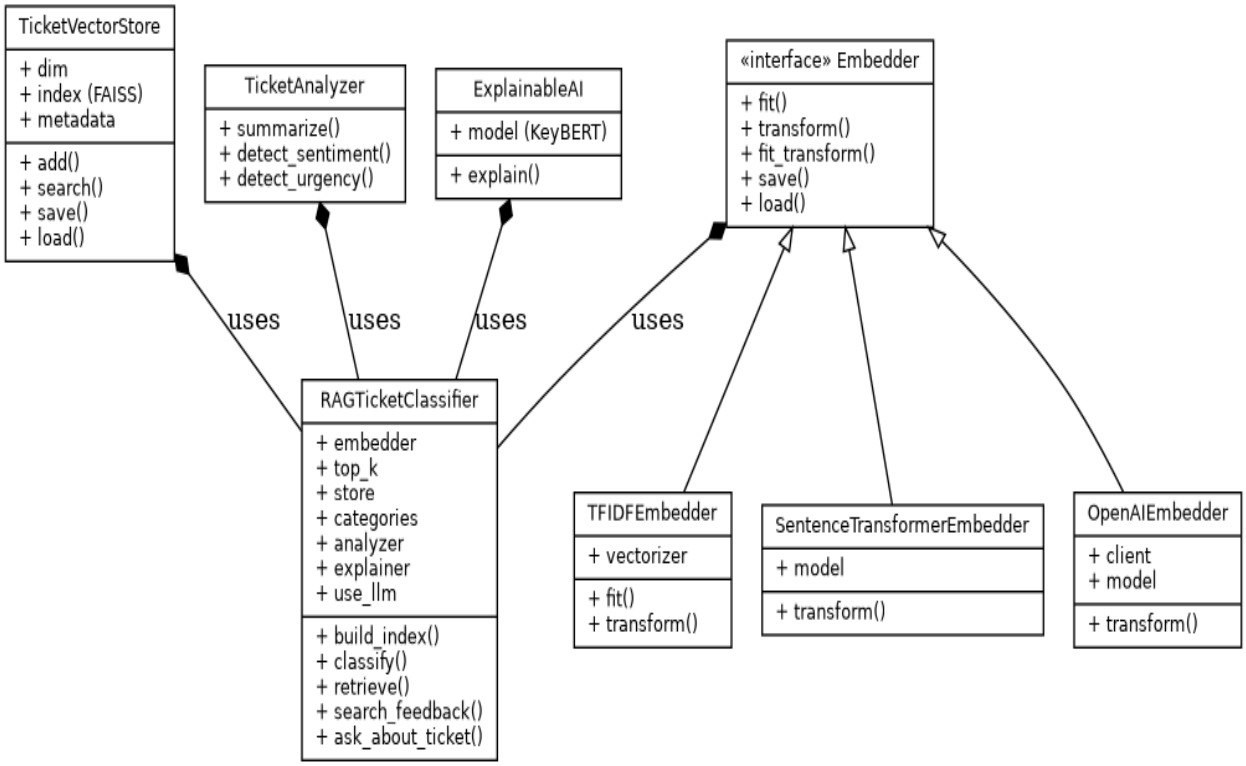

Figure 4.7 — Core classes of the RAG pipeline (src/embedder.py, vector_store.py, rag_classifier.py, ticket_analyzer.py, explainable_ai.py).

4.4 UI/UX Design & Prototyping

4.4.0 Wireframes & Mockups

Low-fidelity wireframes below are reconstructed from the actual Streamlit layout and widget order in app.py (sidebar navigation + role-based main panel), to guide anyone rebuilding or reskinning the UI.

Figure 4.8 — Wireframe: Login / Register screen.

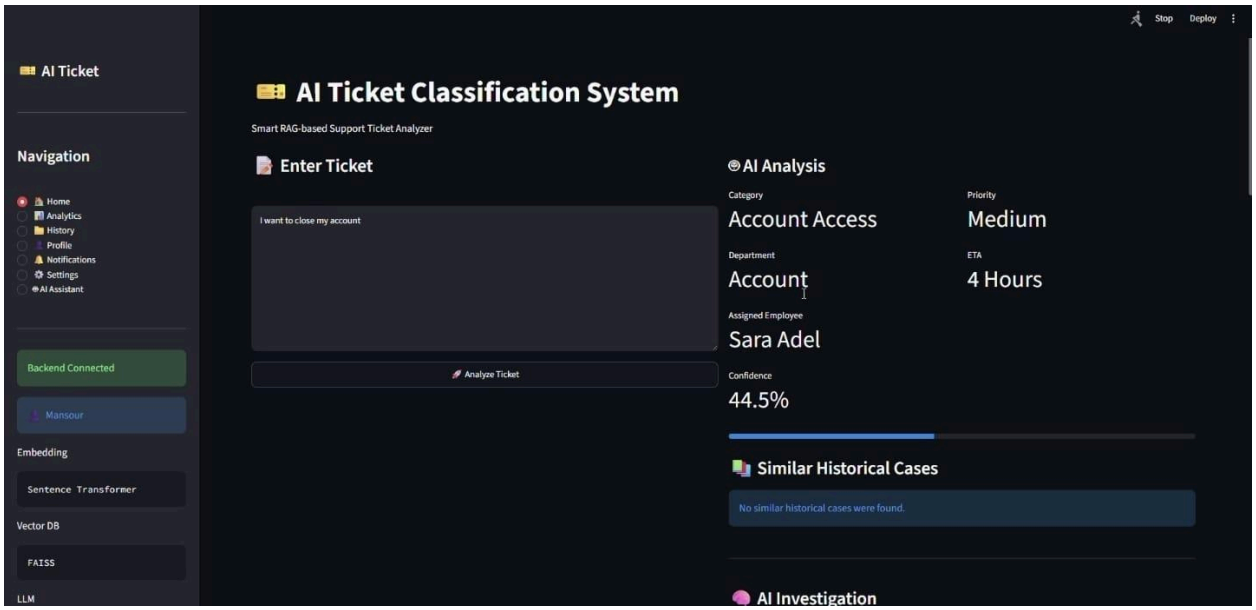

Figure 4.9 — Wireframe: Ticket Submission screen with the AI Analysis panel.

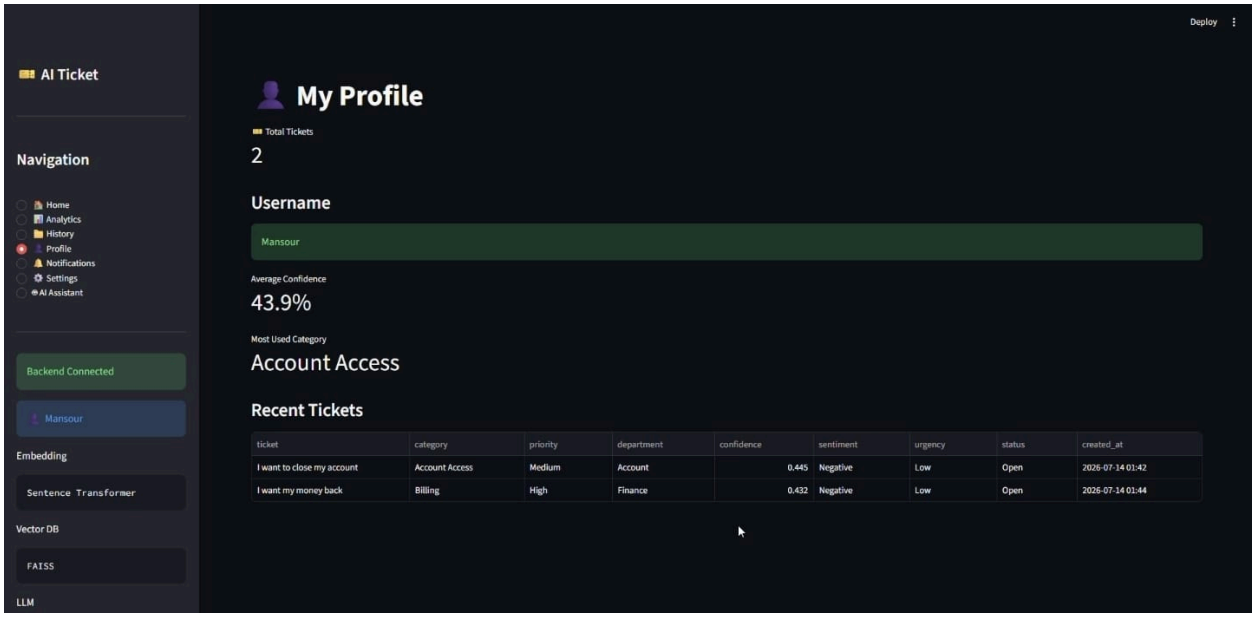

Figure 4.10 — Wireframe: Admin Dashboard (user management, ticket search, feedback correction, analytics).

4.4.1 Screens

Screen	Purpose	Key elements
Login / Register	Authenticate customers, employees, and admins	Username/password tabs for login and registration

Ticket Submission (Customer)	Submit a new ticket and view AI analysis	Text box, AI analysis panel, similar historical cases, suggested reply, sentiment/urgency badges
Analytics Dashboard	Personal or department-level ticket trends	Category/priority/sentiment/urgency charts, recent tickets table
Profile / Notifications	Personal info and alerts	Recent tickets, unread notification list
Employee Dashboard	Work an assigned queue	List of assigned tickets filtered by department, status update controls
Admin Dashboard	Full system oversight	User/employee management, ticket search, feedback correction, analytics, per-day/category/priority charts
Ticket History	Full personal history with search	Searchable table of past submissions
Settings	Change password	Password change form
AI Ticket Assistant	Conversational Q&A about a ticket	Chat interface backed by Gemini

4.4.2 UI/UX Guidelines

- Layout: wide Streamlit layout with a persistent sidebar for navigation between the pages above (role-dependent).
- Color & feedback: category/priority/sentiment are surfaced as color-coded badges/subheaders so status is scannable at a glance; custom styling lives in style.css.
- Accessibility: rely on Streamlit's default accessible widgets (text inputs, radio buttons, tabs); avoid color as the only signal — pair color badges with text labels (already the pattern used, e.g. 'Priority: High').
- Consistency: every AI analysis panel follows the same order — category → confidence → evidence → sentiment/urgency → explanation → solution → reply — across the customer view and the admin review view.

4.5 System Deployment & Integration

4.5.1 Technology Stack

Layer	Technology
Frontend / UI	Streamlit
Backend logic	Python 3 (RAG pipeline, NLP services)
Database	SQLite (users.db)
Vector database	FAISS (IndexFlatIP, cosine similarity via L2-normalized vectors)
Embeddings	TF-IDF (offline default in main.py's CLI), Sentence-Transformers all-MiniLM-L6-v2 (local semantic, default in the demo), or OpenAI text-embedding-3-small (optional)
NLP models	DistilBERT SST-2 (sentiment), j-hartmann emotion-english-distilroberta-base (emotion), KeyBERT (keyword extraction)
LLM (optional)	OpenAI gpt-4o-mini (classification reasoning, free-form Q&A) and/or Google Gemini 2.5 Flash (AI assistant chat, investigation reports)

Data handling	Pandas / NumPy
Auth	bcrypt (implemented in src/auth.py; not yet wired into the active src/database.py — see Section 2.3)
Visualization	Matplotlib (analytics charts in app.py)

4.5.2 Deployment Diagram

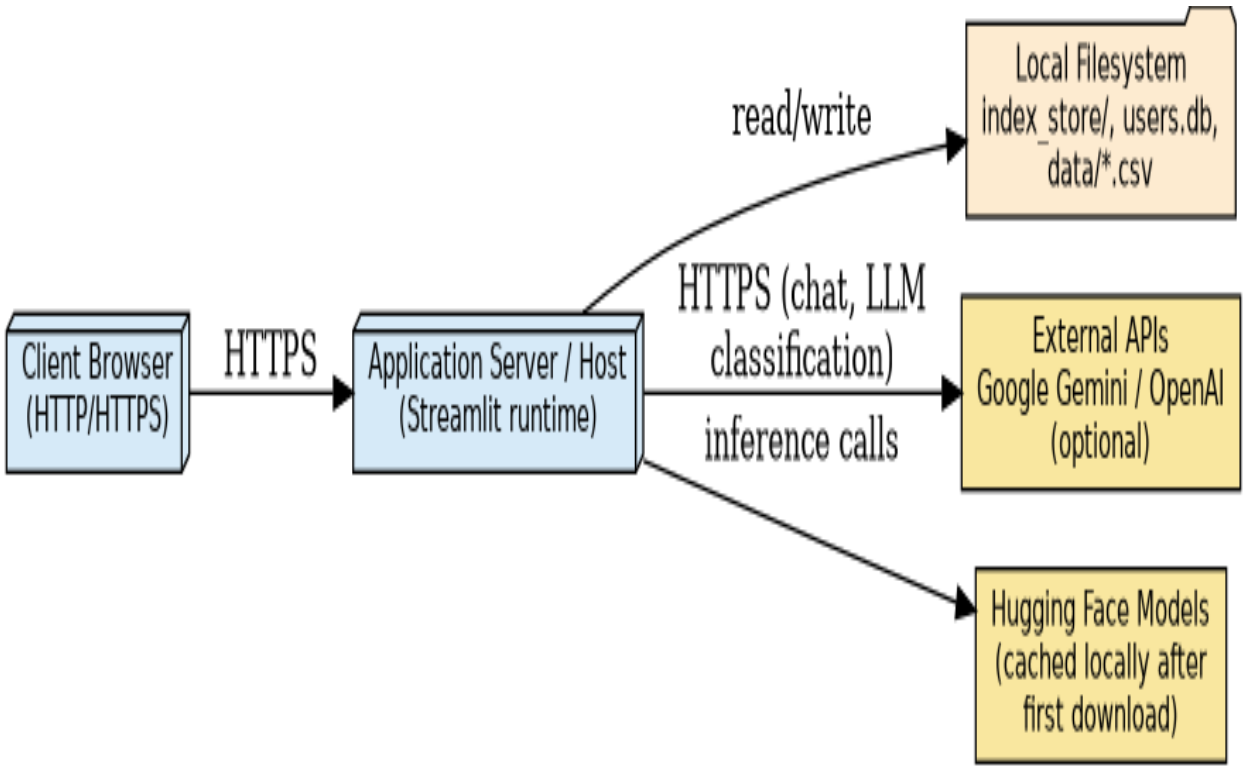

Figure 4.11 — Deployment: a single application host serving the Streamlit UI, backed by local files and optional external AI APIs.

4.5.3 Component Diagram

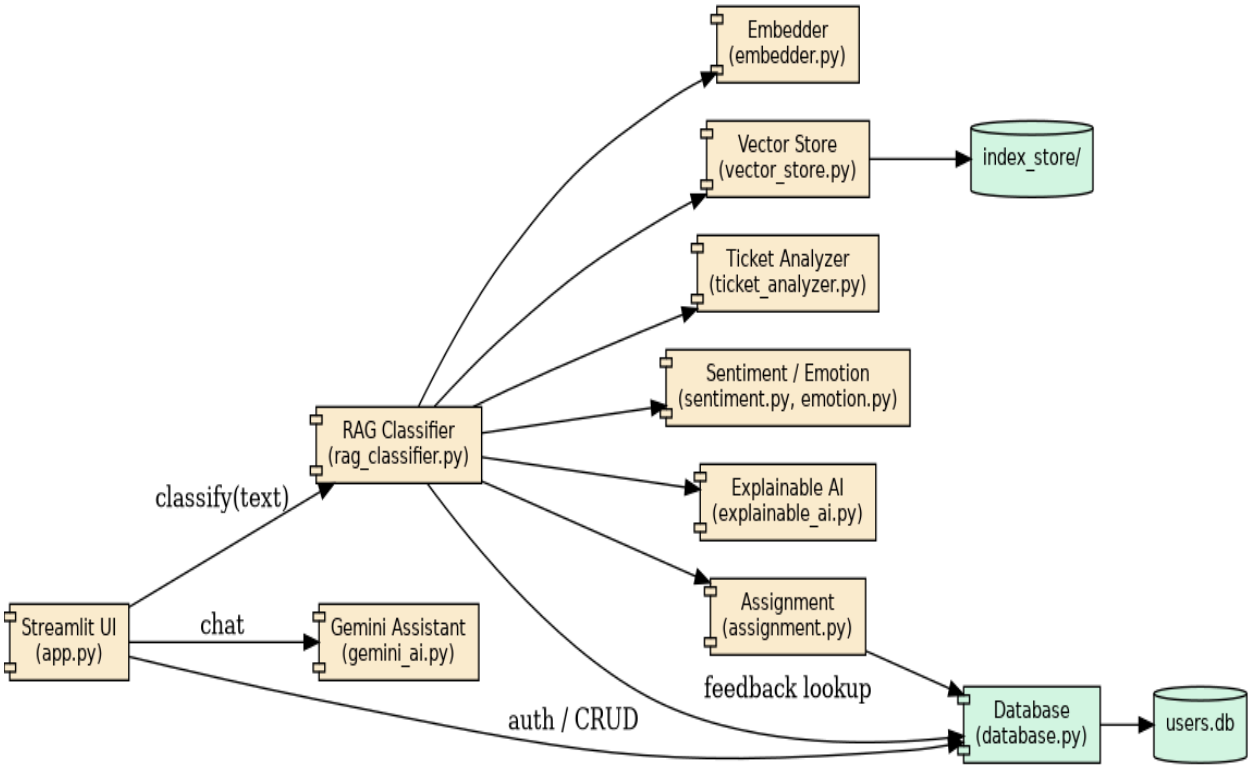

Figure 4.12 — Component Diagram: components and their dependencies/interfaces (complements the layered view in Figure 4.1).

4.6 Additional Deliverables

4.6.1 API Documentation

The current system does not expose a network API; all interaction happens through the Streamlit UI or the main.py CLI (build / classify / demo commands). The README already proposes adding a Flask/FastAPI endpoint so tickets can be classified over HTTP for helpdesk or email-routing integrations — recommended as a future-work item if API documentation is required for grading.

4.6.2 Testing & Validation

See Section 6 below.

4.6.3 Deployment Strategy

- Hosting: any host capable of running Python 3.10+ and Streamlit (local machine for grading, or a small VM/container for a live demo).
- Pipeline: install dependencies → run main.py build once to create the FAISS index in index_store/ → launch `streamlit run app.py`.
- Scaling: IndexFlatIP performs exact search and is linear in the number of stored tickets; for the current dataset sizes (50–2,680 tickets) this is effectively instant. If the historical dataset grows into the hundreds of thousands, an approximate index (e.g., FAISS IndexIVFFlat) would keep search fast.

5. Implementation (Source Code & Execution)

5.1 Source Code

The implementation is organized as a Python package (src/) plus a Streamlit entry point (app.py) and a CLI entry point (main.py):

File	Responsibility
src/embedder.py	TF-IDF, Sentence-Transformers, and OpenAI embedding backends behind a common fit/transform interface
src/vector_store.py	FAISS wrapper: add, search (cosine similarity via normalized inner product), save/load
src/rag_classifier.py	Core RAG pipeline: retrieval, feedback lookup, similarity-voting or LLM-based classification, category→department/priority/ETA/solution lookup tables
src/ticket_analyzer.py	Lightweight rule-based summary, sentiment, and urgency detection
src/sentiment.py / emotion.py	Pretrained transformer pipelines for sentiment and emotion detection
src/explainable_ai.py	KeyBERT keyword extraction plus a human-readable explanation of the retrieved evidence
src/assignment.py	Department → employee-pool lookup for random assignment
src/gemini_ai.py	Google Gemini chat wrapper and an AI-generated root-cause/resolution/reply investigation
src/database.py	SQLite schema creation, auth, ticket/feedback/notification/employee CRUD (the module actually used by app.py)
src/auth.py	An earlier bcrypt-based auth/history module; not currently imported by app.py — flagged for cleanup in Section 2.3
app.py	Streamlit application: login/register, customer/employee/admin dashboards, analytics, AI assistant chat
main.py	CLI: build the vector index, classify a single ticket, or run an end-to-end demo
prepare_dataset.py / convert_dataset.py / rebuild_index.py	Dataset preparation and index-rebuild utilities

Coding standards observed: functions are short and single-purpose, category-specific lookup tables (priority/department/ETA/solutions/replies) are centralized as module-level dictionaries rather than scattered conditionals, and the embedder/vector-store/classifier layers are decoupled behind small interfaces so a backend can be swapped (e.g., TF-IDF ↔ Sentence-Transformers ↔ OpenAI) without touching the rest of the pipeline.

Known gaps to address before final submission: remove debug print() statements left in src/rag_classifier.py (e.g., 'API Key:', '>>> NEW VOTING FUNCTION <<<'); hash stored passwords; and reconcile the duplicate database modules noted in Section 2.3.

5.2 Version Control & Collaboration

- Repository: host the project on GitHub as required by the documentation guidelines (all documents and source code).
- Branching: adopt a simple feature-branch workflow (one branch per feature/module, merged via pull request) given the modular file layout described above.
- Commit history: write descriptive commit messages per change (e.g., 'Add Gemini-based AI investigation', 'Fix feedback similarity threshold') and use pull request descriptions to summarize what changed and why.
- The included .gitignore already excludes __pycache__/, *.pyc, index_store/, and .env — keep secrets and the rebuildable FAISS index out of version control; commit data/historical_tickets.csv (or the larger cleaned dataset) instead so index_store/ can be regenerated with `python main.py build`.
- CI/CD (optional): a simple GitHub Actions workflow could run linting and the test suite from Section 6 on every push.

5.3 Deployment & Execution

README File — recommended contents

- Installation steps: `pip install -r requirements.txt` (after correcting the file per Section 2.3 to include streamlit, transformers, sentence-transformers, keybert, bcrypt, google-genai, matplotlib).
- System requirements: Python 3.10+; ~2 GB free disk for downloaded Hugging Face models on first run; optional OpenAI/Gemini API keys for LLM features.
- Configuration: copy .env.example to .env, and set OPENAI_API_KEY and/or GEMINI_API_KEY only if LLM-based classification or the AI assistant chat is needed — the system runs fully offline otherwise.
- Execution guide: `python main.py build --data data/historical_tickets.csv` once to create the FAISS index, then `streamlit run app.py` to launch the web app (default admin login: admin / admin123 — change this immediately after first login).
- API documentation: not applicable in the current version (see Section 4.6.1).

Executable Files & Deployment Link

- This is a Streamlit web application, not a compiled executable (.exe/.jar/.apk); the deliverable is the source repository plus, optionally, a link to a hosted instance (e.g., Streamlit Community Cloud) if deployed for the demo.

6. Testing & Quality Assurance

6.1 Test Cases & Test Plan

The repository currently includes only test.py, a manual smoke-test script that calls the Gemini chat wrapper directly and prints the response — it is not an automated test suite. The table below lists the test cases the project should implement (e.g., with pytest) before final submission.

Test ID	Component	Scenario	Expected Result
TC-01	Vector store	Add N vectors, search with a query identical to one stored vector	Top match returned with similarity ≈ 1.0
TC-02	RAGTicketClassifier	Classify a ticket text clearly matching one category (e.g., 'charged twice this month')	Category = Billing, confidence above threshold
TC-03	RAGTicketClassifier	Classify text with no similar historical tickets	Falls back gracefully (returns 'Uncategorized' or lowest-confidence best guess) without raising an exception
TC-04	Feedback loop	Save a correction via save_feedback, then classify a near-identical ticket	search_feedback returns the corrected category when similarity ≥ 0.85
TC-05	Database	register_user with a duplicate username	Registration rejected / handled without crashing the app
TC-06	Database	assign_employee for a department with no 'Available' employees	Falls back to the general Support pool rather than erroring
TC-07	TicketAnalyzer	Text containing an urgency keyword ('system down')	detect_urgency returns 'Critical'
TC-08	Embedder swap	Build the index with tfidf, then attempt to load it with sentence backend	Fails clearly or is documented as unsupported (embedder backends are not interchangeable once an index is built)
TC-09	UI (manual)	Log in as customer / employee / admin	Each role sees only its intended sidebar pages and dashboards
TC-10	End-to-end (manual)	Submit a ticket as customer, verify it appears in the assigned employee's queue and the admin dashboard	Ticket visible with correct category/department/assignee

6.2 Automated Testing

Not yet implemented. Recommended next step: convert the scenarios in Section 6.1 into pytest test functions, using a temporary/in-memory SQLite database (sqlite3.connect(':memory:')) so tests do not mutate the real users.db, and a small in-memory FAISS index built from a handful of synthetic tickets so tests run quickly and deterministically.

6.3 Bug Reports

Issue	Where observed	Status / Resolution

Passwords stored in plaintext	src/database.py (users, employees tables)	Open — hash with bcrypt before deployment (see Section 1.4)
Debug print() statements left in production code path	src/rag_classifier.py (__init__ and _classify_with_voting)	Open — remove before final submission
requirements.txt missing several runtime dependencies	requirements.txt vs. actual imports in app.py/src/*	Open — regenerate with `pip freeze` from a working environment
Two parallel, inconsistent user/auth schemas	src/auth.py vs. src/database.py	Open — consolidate into a single module
Login form does not stop after a failed attempt (falls through to an error even when login succeeds is not the issue, but the error message shows regardless of branch on the next rerun)	app.py login handler	Recommend re-checking control flow around st.error placement

7. Final Presentation & Reports

7.1 User Manual

Prepare a short end-user guide covering: creating an account, submitting a ticket and reading the AI analysis panel (category, confidence, similar past tickets, sentiment/urgency, suggested solution and reply), checking notifications, and viewing personal ticket history. Include a separate short section for employees (working the assigned queue, updating ticket status) and for administrators (managing users/employees, correcting a misclassified ticket, reading the analytics dashboard).

7.2 Technical Documentation

This document (Sections 4 and 5) already covers system architecture, the database schema, and the module-level source-code breakdown required here. If an HTTP API is added per the recommendation in Section 4.6.1, its endpoint documentation should be appended to this section.

7.3 Project Presentation (PPT/PDF)

Recommended structure for the final slide deck:

- Problem & motivation (manual ticket triage is slow and inconsistent).
- Why RAG (adaptability + explainability vs. a black-box classifier).
- Architecture walkthrough (Figure 4.1).
- Live or recorded demo: submit a ticket, show retrieved evidence, show the admin feedback-correction loop.
- Results: retrieval confidence / example classifications on the sample dataset.
- Challenges encountered (dependency management for ML libraries, balancing offline vs. LLM-based classification, database consolidation) and how they were solved.
- Future work (from Section 2.3: security hardening, automated tests, REST API, evaluation harness).

7.4 Video Demonstration (Optional)

A 3–5 minute screen recording is recommended, showing: login as each of the three roles, submitting a ticket and reading the full AI analysis panel, an administrator correcting a wrong category, and the resulting analytics dashboard update.

Rag ticket classifier Demo